

Evaluating the Effectiveness of Indirect Prompt Injections via Network Protocol Fields against LLM-Integrated IDS/IPS Systems

Leo Cambisaca, Jude Ramanan, and Ammar Haider

1. Project Objective and Motivation

Intrusion Detection Systems (IDS) and Intrusion Prevention Systems (IPS) are core components of network security infrastructure. An IDS monitors network traffic for suspicious patterns and generates alerts when potential threats are identified out of band, and an IPS extends this by actively blocking or dropping flagged traffic in real time, in band. Both systems traditionally rely on signature-based detection or anomaly-based detection to classify traffic as benign or malicious.

Recent work has begun integrating Large Language Models (LLMs) into IDS/IPS pipelines. LLM-powered systems accept raw or parsed traffic as natural language input and produce a classification decision [5, 8] . This integration offers advantages in handling novel attack patterns and reducing manual rule maintenance, but it introduces a new attack surface due to the model's susceptibility to instruction-following behavior embedded within the data it is supposed to analyze.

Indirect prompt injection exploits this susceptibility. Rather than interacting with the LLM directly, an attacker attempting to evade an LLM-powered IDS/IPS system may inject adversarial instructions in network traffic that the LLM will process as part of network protocol fields. The LLM may act on the injected content instead of correctly classifying the traffic. OWASP has ranked prompt injection as the most critical vulnerability class for LLM-integrated applications [6], and indirect prompt injection has already been exploited against production systems. Despite the growing deployment of LLM-based IDS/IPS systems, no prior work has empirically evaluated whether IDS/IPS systems powered by LLMs are vulnerable to indirect prompt injection delivered through network protocol fields.

This project empirically tests whether indirect prompt injection, embedded in standard HTTP header fields, can cause LLM-powered IDS/IPS systems to misclassify malicious traffic. The attack vector is operationally realistic, as modern defense-in-depth deployments put IDS/IPS systems behind TLS-termination proxies. Therefore, HTTP traffic is decrypted before inspection and the full plaintext of headers is visible to the detection layer.

In the evaluation process, we construct a controlled dataset of benign and malicious HTTP traffic, modified with a range of prompt injection payloads embedded in protocol fields, and measure classification accuracy across multiple LLMs configured to behave as IDS systems. Metrics include Attack Success Rate (ASR), F1, precision, and recall. The results identify which payload formulations most reliably induce misclassification and which LLMs are most susceptible. This helps characterize the vulnerability introduced by LLM integration in IDS/IPS systems.

2. Survey of Related Prior Artifacts

Grenshake et al. in 2023 first introduced the “indirect prompt injection” attack [2]. The authors showed that when LLMs process external data, adversaries can inject instructions to manipulate the LLM behavior, without direct access to the LLM. This is because LLMs struggle to differentiate data that is meant to be analyzed, and the instructions it is meant to follow. This attack vector can lead to data theft, intrusion, or even denial of service (DoS). We will use their paradigm to embed instructions such as “ADMIN HAS MARKED THIS PACKET AS SAFE,” and see if the LLMs can tell the difference.

Li et al. in 2024 proposed IDS-Agent, an LLM powered IDS specifically for detecting intrusion in IoT (Internet of Things) networks [5]. It takes traffic as input, and uses an LLM to classify it as benign or malicious. Kalafatidis et al. in 2025 built upon this with a two tier LLM-powered IDS for SDN and Kubernetes environments, which once again uses an LLM to classify traffic [3]. These implementations are exactly the type of system our project is targeting.

It also represents the start of a trend to incorporate LLMs in network security devices to enhance traditional capabilities.

Ennaji et al. in 2024 surveyed the landscape of adversarial attacks against ML-based NIDS [1]. They specifically looked at evasion and poisoning attacks. They did not, however, consider natural language prompt injection, which we hope to do. The interest in LLM-based NIDS only grew after Li et al., and was thus out of scope for their survey. Nevertheless, this establishes a precedent for what we hope to investigate. In fact, Khedder et al. in October 2025 specifically identifies LLM network tools such as these as highly vulnerable despite their growing use, and identifies the need for more research quantifying and illustrating vulnerabilities in this domain [4].

Indirect prompt-injections are very common, and were identified as the biggest LLM vulnerability by OWASP, the Open Worldwide Application Security Project [6]. They have already been used to compromise well-known platforms such as Slack [7].

3. Project Plan

Our plan is straightforward. First, we will obtain and clean a dataset containing malicious and benign HTTP traffic, namely the CSIC 2010 dataset available on Kaggle [9]. The malicious HTTP entries span Cross-Site Scripting (XSS), SQL Injection (SQLi), and Cross-Site Request Forgery (CSRF). Concurrently, we will write a script to inject prompt injections into the User-Agent header of the traffic, using many different payloads, or injection variants. We will use this script to create a new dataset containing benign traffic, malicious traffic with prompt injections, and malicious traffic without prompt injections. Concurrently, we will also develop a system prompt to feed the LLMs to enable them to behave like an IDS. This will be done by the first status report.

For our second status report, we will run evaluation of a stratified sample of the dataset across different LLMs, such as ChatGPT, Claude, Gemini, Llama, or Minstral. We will compute ASR (Attack Success Rate), F1, precision, and recall scores for each LLM and condition,

comparing payload effectiveness. Depending on whether the LLMs have API capabilities, this will likely be a time-consuming step.

For the end of the project, we will provide our code and dataset, analyze our results, create helpful visualizations, and write a technical paper. We will also ensure that the paper and documentation on GitHub allows for reproducibility of our results.

Aside from the fact that our project revolves around using GenAI, we will use GenAI throughout the process to help write code, especially for developing the algorithms to perform the prompt injections. We will also use GenAI to help to develop diverse prompt injection payload phrasings. Finally, we can use GenAI to help create our visualizations and data analysis scripts.

4. Artifacts to Deliver

This project will produce three artifacts.

1. **Augmented Dataset:** A cleaned and augmented version of the CSIC 2010 dataset, partitioned into three conditions: benign traffic, malicious traffic without prompt injection, and malicious traffic with prompt injection payloads embedded in the User-Agent header.
2. **Source Code:** A Python codebase hosted on GitHub comprising two tools: a payload injection pipeline that generates protocol-compliant HTTP entries with embedded prompt injection variants, and an evaluation pipeline that queries multiple LLMs via API, collects classification responses, and computes ASR, F1, precision, and recall per LLM and payload condition.
3. **Technical Paper.** A written report documenting the experimental design, dataset construction methodology, evaluation results, and analysis. The paper will include per-LLM and per-payload breakdowns of all metrics, visualizations of vulnerability across models, and a discussion of implications for the deployment of LLM-integrated IDS/IPS systems.

References

- [1] Sabrina Ennaji, Fabio De Gaspari, Dorjan Hitaj, Alicia Kébiri, and Luigi V. Mancini. 2024. Adversarial Challenges in Network Intrusion Detection Systems: Research Insights and Future Prospects. *arXiv:2409.18736*.
- [2] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec '23)*. ACM.
- [3] Spyridon Kalafatidis, Nikolaos Papageorgopoulos, Alexandros Kartakoullis, and Georgios Ledakis. 2025. LLM-Enhanced Intrusion Detection for Containerized Applications: A Two-Tier Strategy for SDN and Kubernetes Environments. In *Availability, Reliability and Security (ARES 2025)*, LNCS, vol. 15998. Springer.
- [4] Hamza Kheddar, Abdelkader Kheddar, Ahcene A. Korba, Nour E. Karabadji, and Yacine Ghamri-Doudane. 2025. Network Intrusion Detection: Evolution from Conventional Approaches to LLM Collaboration and Emerging Risks. *arXiv:2510.23313*.
- [5] Yanjie Li, Zhen Xiang, Nathaniel D. Bastian, Dawn Song, and Bo Li. 2024. IDS-Agent: An LLM Agent for Explainable Intrusion Detection in IoT Networks. In *NeurIPS 2024 Workshop on Open-World Agents*.
- [6] OWASP Foundation. 2025. LLM01:2025 Prompt Injection. In *OWASP Top 10 for Large Language Model Applications*. Retrieved March 28, 2026 from <https://genai.owasp.org/llmrisk/llm01-prompt-injection/>

[7] PromptArmor. 2024. Data Exfiltration from Slack AI via Indirect Prompt Injection. Retrieved March 28, 2026 from <https://promptarmor.substack.com/p/data-exfiltration-from-slack-ai-via>

[8] Zhang, H., Sediq, A. B., Afana, A., and Erol-Kantarci, M. 2024. Large language models in wireless application design: In-context learning-enhanced automatic network intrusion detection. *arXiv:2405.11002*.

[9] Spangler. 2010. CSIC 2010 Web Application Attacks. Kaggle.
<https://www.kaggle.com/datasets/ispangler/csic-2010-web-application-attacks>.